Foodie - Online Food Delivery Platform

Java Full Stack Capstone Project

Angular 18 | Spring Boot 3 | MySQL | Docker | Jenkins | AWS EC2

Repository: https://github.com/Manoj123312/glvc-capstoneproject (branch: capstone-project)

1. Introduction

Foodie Inc. aggregates restaurants of a city and their famous dishes on a central system. This project is a proof of concept of an online food ordering platform. Users can browse restaurants and dishes, filter them by preference, place orders and track their order history. Administrators manage users, restaurants, dishes and the order lifecycle.

1.1 Objectives

• Build a responsive Angular front end for the Admin and the End User.

• Build a Spring Boot REST back end with entities, services, controllers, Swagger documentation and JUnit tests.

• Design a MySQL schema and persist data using JPA/Hibernate.

• Secure the system with JWT-based authentication and role-based authorisation (ADMIN / USER).

• Containerise with Docker and automate build and deployment with a Jenkins pipeline on AWS EC2.

1.2 Scope and roles

Role	Capabilities
Admin	Login; user management (change role, enable/disable, delete); restaurant management (add/edit/delete); product (dish) management; view all orders and update order status.
User	Register/login; browse restaurants and dishes; filter by restaurant, category, veg/non-veg, max price and text; add to cart; buy (place order); order history with status; cancel an order while Placed/Confirmed; edit profile.

2. Technology Stack

Layer	Technology	Purpose
Front end	Angular 18 (standalone components, signals, lazy routes)	SPA UI, route guards, HTTP interceptor
Back end	Java 17, Spring Boot 3.3	REST API
Persistence	Spring Data JPA / Hibernate	ORM and repositories
Database	MySQL 8 (H2 for zero-setup local run)	Data store
Security	Spring Security + JWT (jjwt), BCrypt	Authentication / authorisation
API docs	springdoc-openapi (Swagger UI)	Interactive documentation
Testing	JUnit 5, Mockito	Service-layer unit tests
DevOps	Docker, Docker Compose, Jenkins, AWS EC2	Containerisation and CI/CD

3. System Architecture

The system is a three-tier application. The Angular SPA calls the Spring Boot REST API over HTTP/JSON. The API is organised in layers (controller, service, repository, entity) and talks to MySQL through JPA. Every protected request carries a JWT in the `Authorization: Bearer` header, added automatically by an Angular HTTP interceptor.

```
Browser (Angular SPA, :4200 / nginx :80)
       |  HTTP + JSON (JWT in Authorization header)
       v
Spring Boot API (:8080)
  Controller -> Service -> Repository -> Entity
  JwtAuthFilter | SecurityConfig (CORS open) | GlobalExceptionHandler | Swagger
       |  JDBC / JPA
       v
```

```
MySQL 8 (foodie_db)   [H2 in-memory when profile = h2]
```

Note: the back end is delivered as one Spring Boot application with clearly separated modules (auth, users, restaurants, dishes, orders). Each module can be split into an independent microservice later without changing the API contract.

4. Database Design

Tables are created automatically by Hibernate (*ddl-auto=update*); *database/schema.sql* contains the equivalent reference SQL.

Table	Key columns	Relationships
users	id, name, email (unique), password (BCrypt), phone, address, role (ADMIN/USER), active	1 user : N orders
restaurants	id, name, cuisine, city, address, image_url, rating	1 restaurant : N dishes
dishes	id, name, description, price, category, veg, image_url, available, restaurant_id	N dishes : 1 restaurant
orders	id, user_id, status, total_amount, delivery_address, created_at	N orders : 1 user; 1 order : N items
order_items	id, order_id, dish_id, dish_name, restaurant_name, unit_price, quantity	Snapshot of the dish at purchase time

Order items store a **snapshot** of the dish name and price so that order history stays correct even if the dish is edited or deleted later. Order status values: PLACED, CONFIRMED, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCELLED.

5. Back-end Design

5.1 Package structure

Package	Contents
entity	User, Restaurant, Dish, OrderEntity, OrderItem (JPA entities)
repository	Spring Data JPA repositories incl. dynamic dish search query
service	AuthService, UserService, RestaurantService, DishService, OrderService (business logic, unit tested)
controller	AuthController, UserController, RestaurantController, DishController, OrderController (Swagger tagged)
security	JwtService, JwtAuthFilter, AppUserDetailsService
config	SecurityConfig (CORS open, role rules), OpenApiConfig, DataSeeder
exception	NotFoundException, BadRequestException, GlobalExceptionHandler
dto	Request/response records with Bean Validation

5.2 REST API

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register a user and return JWT
POST	/api/auth/login	Public	Login and return JWT
GET	/api/restaurants, /api/restaurants/{id}	Public	List / view restaurants
POST/PUT/DELETE	/api/restaurants[/{id}]	ADMIN	Manage restaurants (delete cascades to its dishes)
GET	/api/dishes?restaurantId&category&veg&maxPrice&q	Public	Filtered dish search

Method	Endpoint	Access	Description
POST/PUT/DELETE	/api/dishes[/{id}]	ADMIN	Manage dishes
GET	/api/admin/users[/{id}]	ADMIN	List / view users
PUT/DELETE	/api/admin/users/{id}	ADMIN	Change role/active, delete (not self)
GET/PUT	/api/users/me	Logged in	View / update own profile
POST	/api/orders	Logged in	Place an order (price calculated server-side)
GET	/api/orders/my	Logged in	Order history of current user
PUT	/api/orders/{id}/cancel	Owner	Cancel while PLACED/CONFIRMED
GET	/api/admin/orders	ADMIN	All orders
PUT	/api/admin/orders/{id}/status	ADMIN	Update order status

5.3 Security

• Passwords are hashed with BCrypt; no plain-text passwords are stored.

• On login a signed JWT (HS256, 24 h) is issued; *JwtAuthFilter* validates it on each request. The secret is read from the *JWT_SECRET* environment variable.

• Role rules: GET restaurants/dishes are public; modifying them and everything under /api/admin/** require ADMIN; all other endpoints need authentication. Unauthenticated requests get 401, wrong role gets 403.

• CORS is open (all origins, methods and headers) as required by the course.

• Order totals are calculated on the server from database prices, so clients cannot tamper with prices.

• A global exception handler returns consistent JSON errors (400 validation, 401, 404, 409).

5.4 Swagger and JUnit

Swagger UI is available at `/swagger-ui.html` with a Bearer-token 'Authorize' button. Unit tests (JUnit 5 + Mockito) cover the services: OrderServiceTest (total calculation, unavailable/unknown dish, cancel rules, status update), DishServiceTest (create, search normalisation, delete, not-found) and UserAuthServiceTest (registration, duplicate email, JWT, self-delete protection, role update). 16 tests, all passing.

6. Front-end Design (Angular)

Area	File(s)	Responsibility
Services	core/services/auth.service.ts, api.services.ts, cart.service.ts	Back-end endpoint calls (HttpClient), session and cart state
Interceptor	core/guards/auth.interceptor.ts	Adds JWT; logs out on 401
Guards	core/guards/auth.guard.ts	authGuard (logged in) and adminGuard (ADMIN)
Auth pages	pages/auth/login, register	Login and registration forms
User pages	pages/user/home, cart, orders, profile	Menu with filters, cart and checkout, order history, profile
Admin pages	pages/admin/admin-restaurants, admin-dishes, admin-users, admin-orders	CRUD and management screens
Shell	app.component.ts, app.routes.ts, app.config.ts	Navigation bar, lazy-loaded routes, providers

The API base URL is derived from the current host (*core/config.ts*), so the same build works on localhost and on EC2.

7. Key User Flows

• **Buy food:** Browse menu -> filter -> Add to cart -> Cart -> confirm address -> Place order -> appears in My Orders as PLACED.

• **Track:** Admin changes status (CONFIRMED, PREPARING, OUT_FOR_DELIVERY, DELIVERED) in Orders; the user sees the updated status.

• **Admin product management:** Admin -> Restaurants / Dishes -> add, edit, delete; changes appear immediately on the public menu.

• **Admin user management:** Admin -> Users -> change role, disable/enable, delete.

8. DevOps and Deployment

Docker: the back end uses a multi-stage Maven build and a JRE image; the front end is built with Node and served by nginx. *docker-compose.yml* starts MySQL, back end and front end together with a MySQL health check.

Jenkins pipeline (Jenkinsfile): Checkout -> Backend tests and package -> Angular production build -> Docker build -> Docker push (Docker Hub) -> Deploy to EC2 over SSH with *docker-compose.prod.yml*.

AWS EC2: install Docker + Compose plugin, open ports 22/80/8080 in the security group, add Jenkins credentials (*dockerhub-creds*, *ec2-ssh-key*), set DOCKER_USER and EC2_HOST in the Jenkinsfile. The app is served at http://<EC2-public-ip>.

Production hardening: set strong JWT_SECRET and MYSQL_ROOT_PASSWORD, change the seeded admin password, restrict CORS origins and add HTTPS.

9. How to Run Locally

Prerequisites: JDK 17+, Maven 3.9+, Node 18.19+/20, Angular CLI 18 (MySQL and Docker optional).

```
git clone -b capstone-project https://github.com/Manoj123312/glvc-capstoneproject.git
cd glvc-capstoneproject

# Terminal 1 - back end (H2 in memory, no DB setup)
cd backend
mvn spring-boot:run          # http://localhost:8080  |  Swagger: /swagger-ui.html

# Terminal 2 - front end
cd frontend
npm install
ng serve                     # http://localhost:4200

# MySQL mode
mvn spring-boot:run -Dspring-boot.run.profiles=mysql

# Everything in Docker
docker compose up --build    # http://localhost

# Unit tests
cd backend && mvn test
```

IDE: IntelliJ - File > Open > backend folder (pom.xml), run FoodieApplication. Eclipse - Import > Existing Maven Projects. VS Code - open the frontend folder.

Demo accounts: **admin@foodie.com / admin123**, **user@foodie.com / user123**.

10. Testing and Verification

Check	Result
JUnit service tests (16)	All pass

Check	Result
Login as admin and user, JWT issued	OK
USER calling /api/admin/**	403 Forbidden
Anonymous call to protected endpoint	401 Unauthorized
Place order, admin updates status	Total 498.00 computed; status DELIVERED
Dish filters (veg, maxPrice, q)	Correct results
Swagger UI and CORS preflight	200 / Access-Control-Allow-Origin returned
Angular production build	Success

11. Conclusion and Future Enhancements

The project delivers a complete, secure, containerised food-delivery proof of concept covering admin and user journeys. Possible enhancements: payment gateway integration, real-time order tracking (WebSocket), restaurant owner role, ratings and reviews, pagination, image upload, Angular unit/e2e tests, splitting into microservices behind an API gateway, and HTTPS with a managed database (AWS RDS).